

Advanced Software Testing and Analysis / Summer Semester 2025

Practical Project Tasks 02: Fuzzing

Prof. Stefan Wagner and Mohamed Ben Salha ✉

✉ mohamed.bensalha@tum.de

Publication: Tuesday, June 01, 2026, 08:00

Submission: Sunday, July 12, 2026, 23:59

Submission Instructions

- This exercise sheet has **2 pages**.
- The submission deadline for all exercise sheets is **Sunday, July 12, 2026, 23:59**.
- Submit your solutions **once** via Moodle before the submission deadline.
- Combine the solutions for **all** exercise sheets into a **single PDF file** and upload it to Moodle.
- Include your fork of the example project and any additional code/scripts as a **single ZIP file** and upload it to Moodle.
- This exercise is to be completed **individually**. Plagiarism will lead to not passing the project and may result in further disciplinary action.
- Grading is on a *pass/not passed* basis. You will receive feedback via Moodle after the submission deadline.
- If you pass, your exam grade will be improved by 0.3.
- Submissions are graded based on the following criteria
 - **Completeness:** All tasks are addressed.
 - **Correctness:** $\geq 80\%$ of tasks are solved correctly.

Support

Make sure to follow each step carefully and check the deliverable checklist before submitting your work. If you encounter any issues, use the Moodle discussion forum to ask questions.

Task 1 Setup Your Project for Fuzzing

Setup `sirius-kernel` for fuzz-testing with JQF. If you run into problems, consult the JQF wiki¹.

- a) Add both the dependency `jqf-fuzz` and the plugin `jqf-maven-plugin` to the Maven `pom.xml`:

```
<dependency>
  <groupId>edu.berkeley.cs.jqf</groupId>
  <artifactId>jqf-fuzz</artifactId>
  <version>2.0</version>
  <scope>test</scope>
</dependency>

<plugin>
  <groupId>edu.berkeley.cs.jqf</groupId>
  <artifactId>jqf-maven-plugin</artifactId>
  <version>2.0</version>
</plugin>
```

- b) Add the `AmountFuzzer` to the folder `src/test/kotlin/sirius/kernel/commons`. You can find the file in the Moodle course.
- c) Build the project with `mvn test-compile` and run JQF with

```
mvn jqf:fuzz -Dclass=sirius.kernel.commons.AmountFuzzer
            -Dmethod=testAmountScientificString
```

Abort the fuzzing when you feel that the tests are "good enough".

Deliverable Checklist

- **PDF:** A screenshot of the terminal output JQF printed. The last block is enough.

Task 2 Create Your Own Fuzz-Tests

Create test cases similar to `AmountFuzzer` for `sirius-kernel`. As described in 1b), you can find the `AmountFuzzer` in the Moodle course. You do not have to achieve 100% code coverage; instead, try to create test cases where you think it makes sense to use fuzzing. Try to create 2-3 test cases for each class. Run the tests with JQF.

- a) Create test cases for `sirius.kernel.commons.Value`
- b) Create test cases for `sirius.kernel.commons.MultiMap`
- c) Create test cases for `sirius.kernel.commons.Trie`

Deliverable Checklist

- **PDF:** A screenshot of each JQF test run.
- **PDF:** A written report summarizing the findings. Explain how you came up with your test cases and how you would interpret the test results.
- **ZIP:** Provide the source code of your **repository fork** as a ZIP file. Ensure the ZIP file includes all project files and your added test cases. You only need to add the repository ZIP once per submission.

¹<https://github.com/rohanpadhye/JQF/wiki/Fuzzing-a-Compiler>